Laporan Tugas Kecil 2

IF2211 Strategi Algoritma

Syaqina Octavia Rizha - 13524088

Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung
2026

Bab 1 : Penjelasan Algoritma

```
void buildOctree(OctreeNode* node, int depth, int maxDepth, vector<Triangle> triangles, vector<Cube> &voxel, vector<Checker> &c) {

    // cek apakah cube ini kena triangle
    bool intersect = false;
    for(auto& tri : triangles){
        if(Intersects(tri, node->box)){
            intersect = true;
            break;
        }
    }

    if(!intersect){ // jika tidak berpotongan, maka tidak perlu lanjut telusuri
        c[depth-1].notChecked++;
        return; // kosong, stop
    }

    if(depth == maxDepth){ // Cek apakah sudah voxel terkecil
        node->isFilled = true; // kalau sudah masukkan voxel ke list
        voxel.push_back(node->box);
        return;
    }

    node->subdivide(); // Kalau berpotongan, tapi belum voxel terkecil -> bagi 8

    for(int i = 0; i < 8; i++){
        c[depth].nodeBuild++;
        buildOctree(node->children[i], depth + 1, maxDepth, triangles, voxel, c);
    }
}
```

1. Karena penggunaan thread. Node awal yang dijadikan parameter pemanggilan fungsi pertama kali, bukanlah Root. Tapi Node (kubus) yang sudah dibagi 8 (childrennya) sehingga depth awal pada parameter adalah 1
2. Kemudian dicek apakah di kubus tersebut berpotongan dengan segitiga (bisa juga kondisi ada segitiga di dalamnya), menggunakan fungsi Intersects()
3. Jika tidak berpotongan, maka node tidak ditelusuri lebih lanjut (dibuang)
4. Jika berpotongan, cek apakah depth sudah maksimum. Jika maksimum, maka node disimpan ke list voxel
5. Jika depth belum maksimum, maka kubus dibagi 8 dengan fungsi subdivide()
6. Panggil kembali fungsi buildOctree untuk semua anak, untuk cek intersect lagi (rekusif)

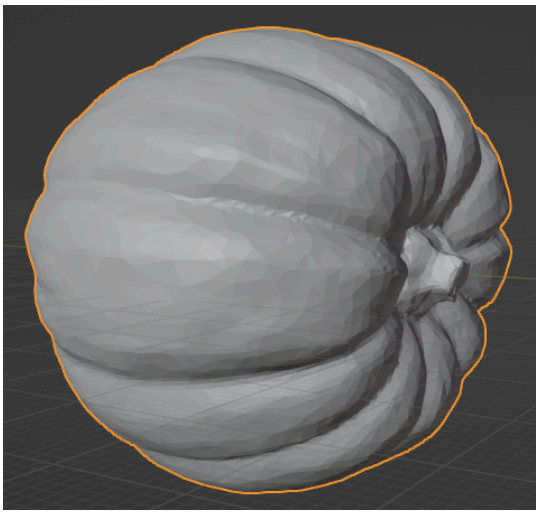
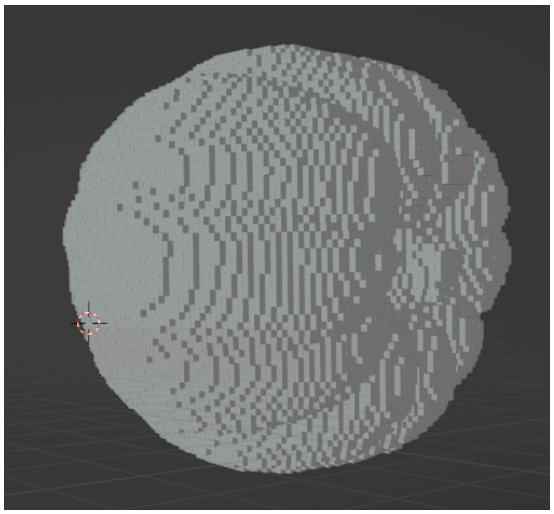
```
void OctreeNode::subdivide(){
    this->isLeaf = false;
    Vertex mid = this->box.center;
    Vertex A = this->box.min; // atasnya A -> E
    Vertex G = this->box.max; // bawahnya G -> C

    this->children[0] = new OctreeNode(A, mid);
    this->children[1] = new OctreeNode({A.x, A.y, mid.z}, {mid.x, mid.y, G.z});
    this->children[2] = new OctreeNode({mid.x, A.y, mid.z}, {G.x, mid.y, G.z});
    this->children[3] = new OctreeNode({mid.x, A.y, A.z}, {G.x, mid.y, mid.z});
    this->children[4] = new OctreeNode({A.x, mid.y, A.z}, {mid.x, G.y, mid.z});
    this->children[5] = new OctreeNode({A.x, mid.y, mid.z}, {mid.x, G.y, G.z});
    this->children[6] = new OctreeNode(mid, G);
    this->children[7] = new OctreeNode({mid.x, mid.y, A.z}, {G.x, G.y, mid.z});
}
```

Kode untuk membagi 1 kubus menjadi 8

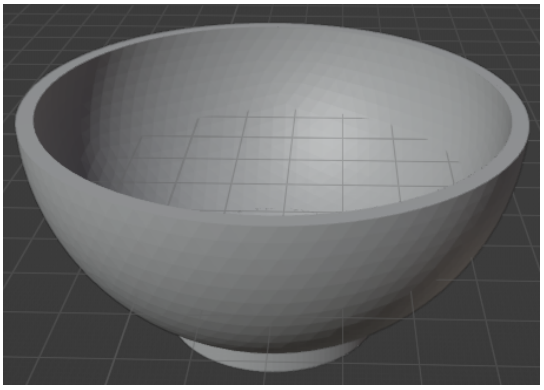
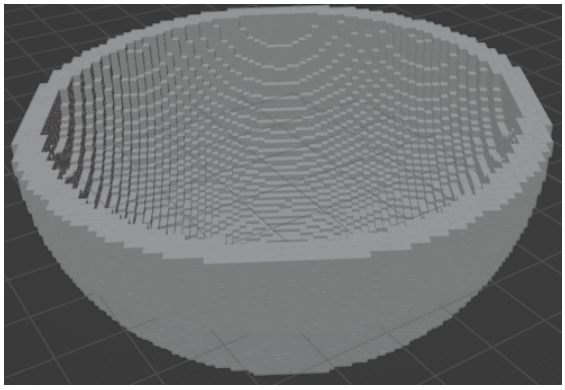
Tambahan: Program ini menggunakan Separating Axis Theorm untuk mengecek apakah kubus berpotongan dengan segitiga

Bab 2 : Testing

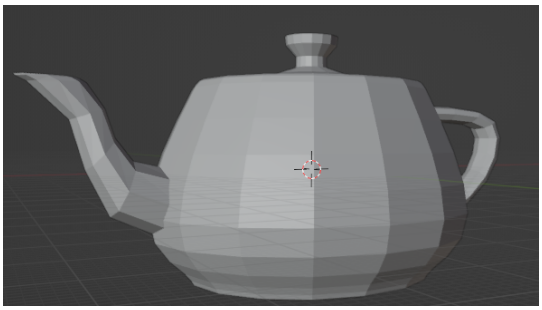
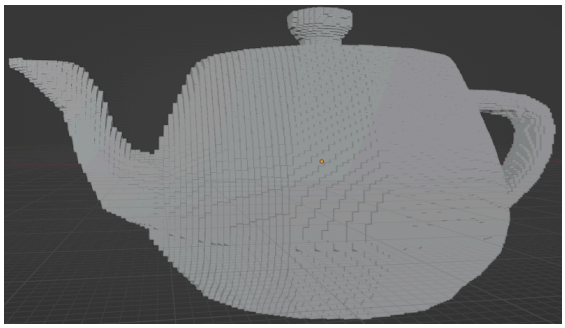
1. pumpkin.obj

Input	Output (depth 6)
	

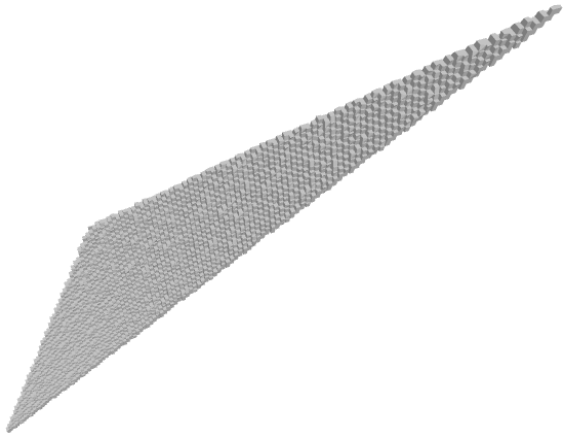
2. bowl.obj

Input	Output (depth 6)
	

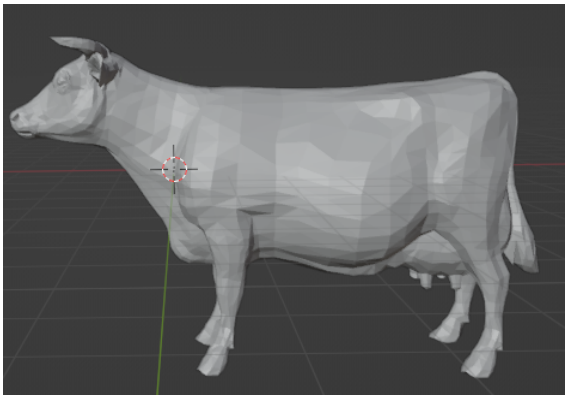
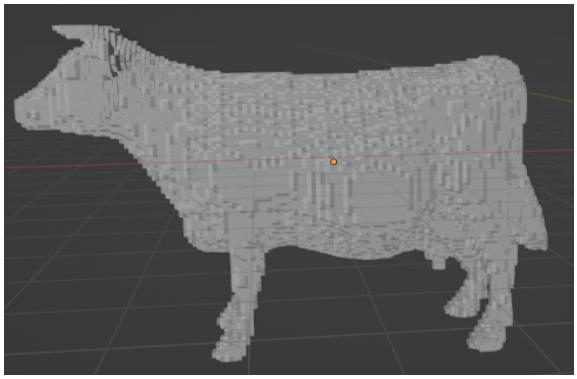
3. teapot.obj (valid input version)

Input	Output (depth 7)
	

4. line.obj

Input	Output (depth 7)
	

5. cow.obj

Input	Output (depth 7)
	

No	Poin	Ya	Tidak
1	Program berhasil di kompilasi tanpa kesalahan	✓	
2	Program berhasil dijalankan	✓	
3	Hasil konversi .obj terdiri dari kubus-kubus kecil	✓	
4	Program informasi-informasi atau report	✓	
5	Program dapat menerima masukan .obj, dan mengembalikan output	✓	
6	[Bonus] Program menggunakan *concurrency*	✓	
7	[Bonus] Program dapat menampilkan visualisasi .obj		✓

Lampiran

Github : https://github.com/SyaqinaOctavia/Tucil2_13524088

Pernyataan Tidak Melakukan Kecurangan

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.

Syaqina Octavia Rizha